StreamHybridSketch: Federated Linear Sketches with Continual-Release Differential Privacy for Streaming Heavy Hitters

Adnan Soni 2023BCD0017
Bhaskara Akshay Sriram 2023BCD0015
Muhammed Minhaj P S 2023BCS0225
Hafiz Feroze 2023BCS0006

***Abstract*—Federated linear sketches with secure aggregation and differential privacy (DP) let a server estimate item frequencies without seeing raw events, but existing methods analyse a fixed dataset in a few rounds. We build an end-to-end streaming pipeline—Kafka ingestion, eight Flower edge clients with pairwise-mask secure aggregation and distributed Gaussian noise, a Flower server, and a Grafana dashboard—and reproduce three base methods in it: DP-HybridSketch, TrieHH and IBLT-based heavy-hitter recovery. On top of the per-window DP-HybridSketch baseline we add tree-based continual release, a *peeling* estimator, adaptive sketch width, a drift detector and an IBLT layer. On Gowalla, Sentiment140 and a drifting synthetic stream, the drift-aware variant recovers heavy-hitter F1 after an abrupt drift in 1.5 instead of 5.5 rounds and cuts upload by 31–61% at equal privacy; tree release with peeling halves the cumulative error at a small budget ($\varepsilon = 0.125$). We also report where the enhancements do *not* help: tree release loses on sliding windows and at $\varepsilon \geq 1$, and the IBLT costs 2–4$\times$ more communication than a dictionary-assisted CountSketch.**

***Index Terms*—federated analytics, count sketch, differential privacy, continual release, secure aggregation, heavy hitters**

I. Introduction

Trending words, busy places and common error codes are *live* statistics over data that should not be centralised. Federated analytics keeps data local and sends only summaries; *linear* sketches such as CountSketch [8] fit well because a server can add them under secure aggregation (SecAgg) and learn only the sum [1], [4]. Real data, however, is an unbounded, drifting stream, and every release spends privacy. Existing federated sketch methods analyse a fixed population in a few rounds [1]–[3].

Problem. Estimate frequencies and top-k heavy hitters over distributed, drifting streams with federated sketches, SecAgg and DP, keeping error, upload and cumulative privacy loss low. This needs a release every window, so a batch job cannot do it, and no single tool covers ingestion, federated orchestration, aggregation, accounting and monitoring.

Contributions. (1) A streaming pipeline with real data flowing Kafka $\rightarrow$ Flower edges $\rightarrow$ SecAgg $\rightarrow$ server $\rightarrow$ Kafka $\rightarrow$ InfluxDB/Grafana, plus an offline harness running the identical protocol code. (2) Reproductions of three base papers. (3) StreamHybridSketch: five ablated stream extensions. (4) A quantitative comparison including negative results.

IOE413 Streaming Data Analytics mini-project, Team 22. Code, data scripts and results: see the accompanying repository and README.

II. Related Work and Gap

Wu et al. [1] (baseline). *Question:* how to size and reuse a CountSketch under secure summation, with and without DP. *Data:* Gowalla, C4, Sentiment140. *Metrics:* ℓ_∞ error, communication, (ε, δ). *Findings:* the needed width depends on the distribution's tail, and a two-phase rule finds it; HybridSketch (shared bucket hash, fresh sign hash per round) beats Shared and Fresh sketches; its DP version is near-optimal. *Model:* none.

Zhu et al. [2] (TrieHH). *Question:* can heavy hitters be found from on-device data with central DP? *Data:* Sentiment140. *Metrics:* precision/recall vs. ε. *Findings:* sampling $\gamma\sqrt{n}$ users per round and keeping trie prefixes with $\geq \theta$ votes is DP without added noise. *Model:* none.

Gascón et al. [3]. *Question:* what does multi-round heavy-hitter recovery cost when clients may only send linear sketches? *Data:* Stack Overflow. *Metrics:* F1 vs. communication. *Findings:* IBLTs [7] with threshold sampling need $\sim$10$\times$ less communication than CountSketch. *Model:* none.

Gap. All three release a fixed number of times. A stream releases every window, so noise or privacy loss accumulates, the right width changes, and windows must react to drift. The tree mechanism [5], [6] handles the first issue for counters but, as we show, conflicts with HybridSketch's fresh signs.

III. System Architecture

Fig. 1 shows the pipeline. A **producer** replays a dataset in event-time order into Kafka, one partition per edge, and marks each window's end with a watermark. Each of 8 **edges** is a Flower client reading only its partition: it samples one item per local user, builds each variant's sketch plus its noise share, masks the payload with pairwise X25519-derived masks and returns it. The **Flower server** plans each round (widths, seeds, noise, IBLT threshold), sums the masked payloads (masks cancel), decodes, and publishes top-k releases to Kafka. An **evaluator**, outside the protocol, scores releases against exact truth and writes metrics to **InfluxDB** for **Grafana**.

IV. Proposed Enhancement: StreamHybridSketch

Privacy model. Each user contributes one item per window, so a depth-d CountSketch has L_2 sensitivity $\sqrt{d}$; the unit is one user in one window (event-level DP [5]). Each of K edges

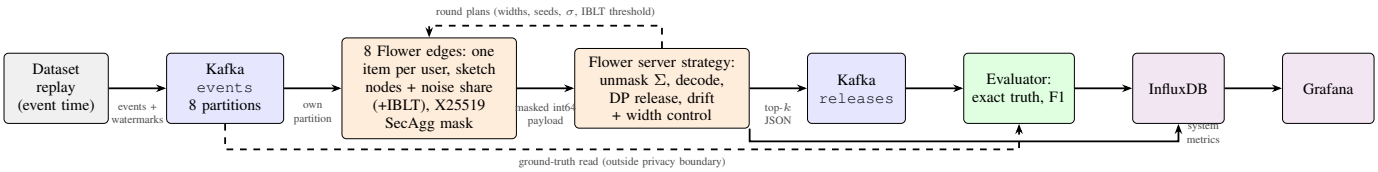


Fig. 1. Pipeline architecture. Raw events never leave the edge; the server sees only the SecAgg sum of noisy sketches. Dashed: the control plane (round plans) and the evaluator's ground-truth read.

adds $\mathcal{N}(0, \sigma^2/K)$, so the server never sees un-noised data. Budgets are tracked with a Rényi-DP [10] accountant.

(a) Tree continual release. Summing t noisy per-window sketches gives noise variance $t\sigma^2$. We instead release the nodes of a binary tree over windows, so any prefix or window is a sum of $O(\log t)$ nodes; each unit appears in $\lceil \log_2 T \rceil + 1$ releases. Upload roughly doubles.

(b) Peeling. Tree nodes must share sign hashes to be summable, which turns HybridSketch into a SharedSketch whose error is dominated by heavy items colliding across rounds. Per node we estimate the top-200 items, subtract them, read every item from the residual and add its own estimate back. This is post-processing, so it is free in privacy.

(c) Adaptive width. The server fits a power law to the previous release's top-20 estimates and applies the two-phase width rule of [1] (power-of-two widths, smoothed over three rounds), at no extra round or privacy cost.

(d) Drift detection. If the top-50 overlap between the newest window and the sliding window falls below half its recent mean, the server restarts the window and resets the width.

(e) IBLT layer. Users threshold-sample their item into an additive IBLT, which the server decodes to list heavy items without a dictionary. It is protected by SecAgg and sampling, not DP.

Algorithm 1 summarises a round; the baseline is the case with one leaf node, fixed W and fresh signs.

From the ablations we recommend two configurations: **SHS-W** (per-round release + (c) + (d)) for window and current-trend queries, and **SHS-C** (tree + peeling + (d), fixed width) for long-horizon totals at small ε.

V. Implementation

Data. *Gowalla*: 6.1M check-ins in 2010 mapped to 5 km grid cells (as in [1]), 295 daily windows, ∼8.5k users each. *Sentiment140*: 1.6M tweets as 22M word events, 124 six-hour windows, ∼9.8k users each. *Synthetic*: Zipf(1.1) over 100k items, 20k users per round, 200 rounds, with the top-200 items replaced at rounds 70 and 140. It stands in for the multi-GB Stack Overflow data of [3].

Tools. Kafka 3.7.0, Flower [9] 1.11.1, Python 3.11, `cryptography` 42, dp-accounting 0.4.4, InfluxDB 1.8, Grafana 10.4, in Docker Compose. **Settings.** Depth 5, baseline width 2048, 8 edges, top-50, 7-round window, $\delta = 10^{-6}$; the candidate dictionary is items seen ≥ 3 times. Unit tests check mask cancellation, IBLT linearity and exact tree sums.

Algorithm 1 One StreamHybridSketch round (window t)

Server: plan $\mathcal{P}_t$: nodes $\mathcal{N}_t$ completed at t (tree) or $\{(0,t)\}$; width W_t; seeds; σ; IBLT threshold
for each edge e **in parallel do**
  consume partition e until watermark t; sample one item per user
  for each node $(\ell, j) \in \mathcal{N}_t$ **do**
    $S_e^{(\ell,j)} \leftarrow$ CountSketch of local items in rounds $[j2^\ell, (j+1)2^\ell)$, width $W^{(\ell,j)}$
    $S_e^{(\ell,j)} \leftarrow S_e^{(\ell,j)} + \mathcal{N}(0, \sigma^2/K)$ ▷ noise share
  end for
  optional: threshold-sample items, encode IBLT B_e
  send $\mathrm{mask}_e([S_e, B_e])$ ▷ $\sum_e \mathrm{mask}_e(x_e) = \sum_e x_e \bmod 2^{64}$
end for
Server: $S \leftarrow \sum_e$ payloads; store nodes; peel IBLT
estimate window / cumulative frequencies from the dyadic cover (with peeling); release top-k to Kafka
drift test (top-k Jaccard) ⇒ reset window; two-phase fit ⇒ W_{t+1}; IBLT load ⇒ threshold

TABLE I
Reproduction of Wu et al. [1]. Single round: median width and fraction of trials with $\ell_\infty \leq \tau$ over 10 targets τ. Multi-round (M=10, W=512): normalised ℓ_∞ ($\times 10^{-3}$).

	Median width (ok)			Multi-round ℓ_∞		
Dataset	minimax	inst.-opt	2-phase	Shared	Fresh	Hybrid
Synthetic	384 (1.00)	48 (0.45)	48 (0.76)	13.58	3.40	3.90
Gowalla	384 (1.00)	24 (0.85)	24 (0.96)	4.69	0.99	1.41
Sentiment140	384 (1.00)	48 (0.72)	64 (0.92)	8.65	1.97	2.68

VI. Experimental Setup and Results

Metrics. *Accuracy*: top-50 F1 against three truths—the 7-window sliding window ($F1_w$), the current window ($F1_n$, which shows drift tracking) and the whole stream so far ($F1_c$)—and the ℓ_∞ error of the estimated counts. *Communication*: upload per edge per round. *Privacy*: (ε, δ) per unit from the accountant. *Drift*: rounds until $F1_n$ regains 90% of its pre-drift level. *System*: round latency and SecAgg cost. Each variant runs on the same stream with the same seeds, so differences come from the method alone.

A. Reproduction of the base papers

For Wu et al. (Table I, Fig. 2), the two-phase rule picks widths 6–16× smaller than the minimax rule and meets the tar-

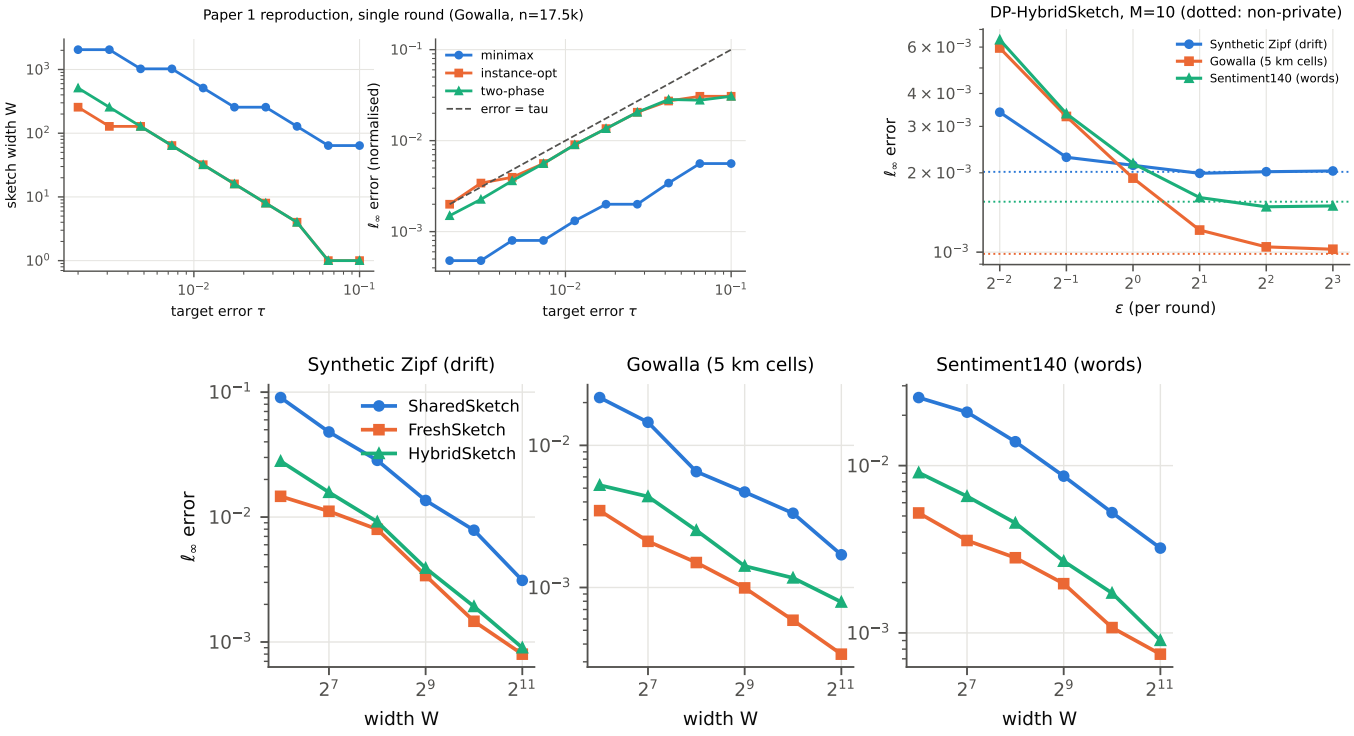


Fig. 2. Reproduction of Wu et al. Top left: single-round width and error of the three width rules vs. target τ (Gowalla). Top right: DP-HybridSketch error vs. ε (dotted: non-private). Bottom: multi-round ℓ_∞ vs. width for Shared, Fresh and Hybrid sketches.

TABLE II
TRIEHH [2] ON SENTIMENT140 (n=659,775 USERS, δ=$1/n^2$): WORDS DISCOVERED, RECALL OF THE TRUE TOP-k AND PRECISION OF THE DISCOVERED SET W.R.T. THE TRUE TOP-200; DP-COUNTSKETCH WITH A PUBLIC DICTIONARY AT THE SAME (ε, δ) FOR REFERENCE.

ε	θ	found	R@50	R@200	P@200	CS R@200
1	15	39	0.76	0.19	1.00	0.93
2	15	80	1.00	0.40	1.00	0.93
4	15	142	1.00	0.70	0.99	0.93
8	15	223	1.00	0.96	0.86	0.93

TABLE III
RECOVERY AFTER THE TWO ABRUPT DRIFTS OF THE SYNTHETIC STREAM (MEAN OF BOTH; F1 VS. CURRENT-ROUND TOP-50, $\varepsilon = 1$).

Variant	pre	min	rec. (rounds)	next 7
Baseline	0.93	0.15	5.5	0.56
SHS-W	0.90	0.60	1.5	0.85
SHS-C	0.91	0.55	1.5	0.84
All components	0.47	0.15	1.0	0.78
All − drift det.	0.49	0.08	7.5	0.24

TABLE IV
SYSTEM COST OF THE LIVE RUN (SYNTHETIC STREAM, 2 S PER WINDOW, 8 EDGE CONTAINERS AND ALL SERVICES ON ONE 12-CORE HOST).

Metric	Value
Rounds (windows) processed	200
Users / events per round	20,000 / 20,000
Variants per round	4
Round latency, median / p95	2.14 s / 2.28 s
Slowest edge: payload build (median)	0.01 s
Slowest edge: wait for watermark (median)	1.56 s
Server SecAgg unmask (median)	1.6 ms
Upload per round, all 8 edges	2.86 MiB
Kafka produce rate (median)	499k events/s

get error in 76–96% of trials, as claimed. HybridSketch beats SharedSketch by 3.2–3.5×, also as claimed, but FreshSketch beat HybridSketch by 1.15–1.4× in our time-ordered rounds, so that part reproduces only partially. DP error reaches the non-private level by $\varepsilon \approx 2$. TrieHH (Table II) finds 39–223 words with precision 0.86–1.00, matching the paper's trend; a DP CountSketch *with* a dictionary reaches recall@200 = 0.93 at any ε.

B. Streaming: baseline vs. enhancements

Overall (Table V, $\varepsilon = 1$). SHS-W matches the baseline's window F1 on Gowalla (0.79 vs. 0.80) with 31% less upload, and uses 2.6× less on the synthetic stream for 0.06 lower F1. On Sentiment140 it loses 0.13 F1 because the detector fires on organic topic shifts. SHS-C matches the baseline on cumulative F1. Enabling every component at once is worse than either configuration, and removing peeling raises cumulative error 1.6–3.5×.

Privacy over time (Table VI, Figs. 5, 4). At $\varepsilon = 0.125$ the tree with peeling cuts final cumulative error 2.0–2.6× and lifts



Fig. 3. Grafana dashboard of the 200-round live run: top-50 F1 per variant against window, current and cumulative truth, and privacy spent. At both drifts the baseline (green) collapses on the current-round panel while SHS-W/SHS-C recover within a round.

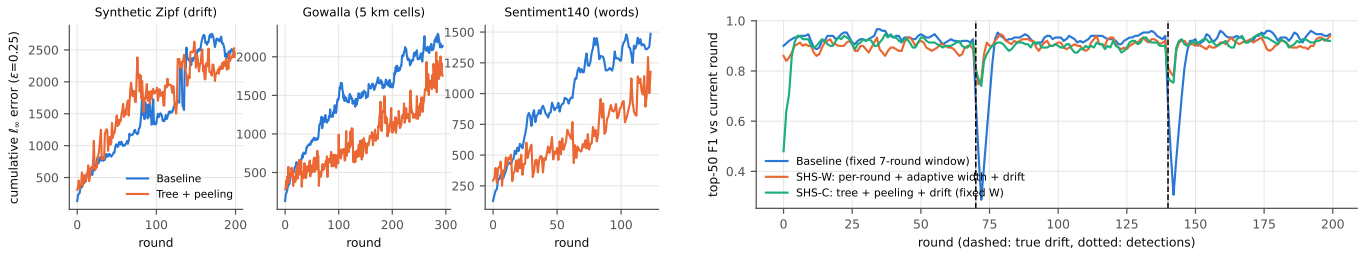


Fig. 4. Left: cumulative ℓ_∞ error over the stream at $\varepsilon = 0.25$, per-round release vs. tree release with peeling. Right: top-50 F1 against the current round's truth on the synthetic stream ($\varepsilon = 1$, 3-round moving average); dashed lines mark the true drifts.

Gowalla's cumulative F1 from 0.74 to 0.90. The gain shrinks to 1.0–1.3× at $\varepsilon = 0.25$, and from $\varepsilon \geq 0.5$ the baseline wins because noise no longer dominates. For short windows the baseline always wins.

Drift (Table III, Fig. 4). The detector fires exactly at rounds 70 and 140. The baseline drops to F1 0.15 and needs 5.5 rounds to recover; SHS-W and SHS-C recover in 1.5, lifting post-drift F1 from 0.56 to 0.85.

Communication (Fig. 6). Adaptive width sits on the fixed-width frontier: it finds a good width without tuning, but does not beat the best one chosen in hindsight. Without DP, the IBLT needs 2–4× more upload than a dictionary-assisted CountSketch on real data; its benefit is needing no dictionary.

C. Live pipeline and system cost

Table IV and Fig. 3 come from a 200-round live run with four variants. With replay paced at 2 s per window, median round latency is 2.14 s, mostly waiting for the watermark (1.56 s); payload building takes 12 ms and SecAgg unmasking 1.6 ms. Replaying the same configuration offline gives identical widths and payload sizes every round and F1 within 0.04.

VII. Discussion

Meaning. No single configuration wins everywhere: window and drift queries favour SHS-W, while long-horizon totals at tight budgets favour the tree, but only with peeling.

Negative results. The tree loses at $\varepsilon \geq 1$ and on windows; combining all components hurts; the IBLT costs more than a dictionary-assisted sketch; FreshSketch beat HybridSketch; the detector raises false alarms on Sentiment140.

Limitations. SecAgg has no dropout recovery; noise seeds are fixed for reproducibility; guarantees are event-level, not user-level; the IBLT is not DP; edges are cross-silo aggregators; streaming results use one seed per setting.

TABLE V

STREAMING RESULTS AT $\varepsilon = 1$, $\delta = 10^{-6}$ PER UNIT OVER THE WHOLE STREAM (TOP-50, 7-ROUND WINDOW, 8 EDGES). $F1_w$/$F1_n$: MEAN TOP-50 F1 AGAINST THE SLIDING-WINDOW / CURRENT-ROUND TRUTH; $F1_c$: CUMULATIVE TOP-50 F1 OVER THE SECOND HALF; ℓ^c_∞: FINAL CUMULATIVE ℓ_∞ ERROR (COUNTS); KIB: UPLOAD PER EDGE PER ROUND.

	Synthetic					Gowalla					Sentiment140				
Variant	$F1_w$	$F1_n$	$F1_c$	ℓ^c_∞	KiB	$F1_w$	$F1_n$	$F1_c$	ℓ^c_∞	KiB	$F1_w$	$F1_n$	$F1_c$	ℓ^c_∞	KiB
Baseline (DP-HybridSketch/round)	0.96	0.91	0.99	1417	80	0.80	0.73	0.98	683	80	0.94	0.84	0.99	460	80
SHS-W (= all − tree, − peel)	0.90	0.90	0.99	11066	31	0.79	0.72	0.96	745	55	0.81	0.75	0.98	686	54
SHS-C (= all − adaptive width)	0.90	0.90	0.99	2725	159	0.57	0.53	0.97	1604	159	0.73	0.69	0.98	1015	157
All components (tree+adapt+drift+peel)	0.63	0.64	0.99	1215	83	0.45	0.43	0.98	977	115	0.67	0.64	0.99	534	172
all − drift detector	0.41	0.40	0.97	3749	48	0.42	0.40	0.94	2090	75	0.86	0.78	0.98	1122	74
all − peeling	0.75	0.76	0.98	2586	87	0.54	0.51	0.98	3440	99	0.71	0.67	0.99	861	191
Non-private reference	0.99	0.92	1.00	2026	80	0.98	0.81	1.00	266	80	0.99	0.85	1.00	360	80

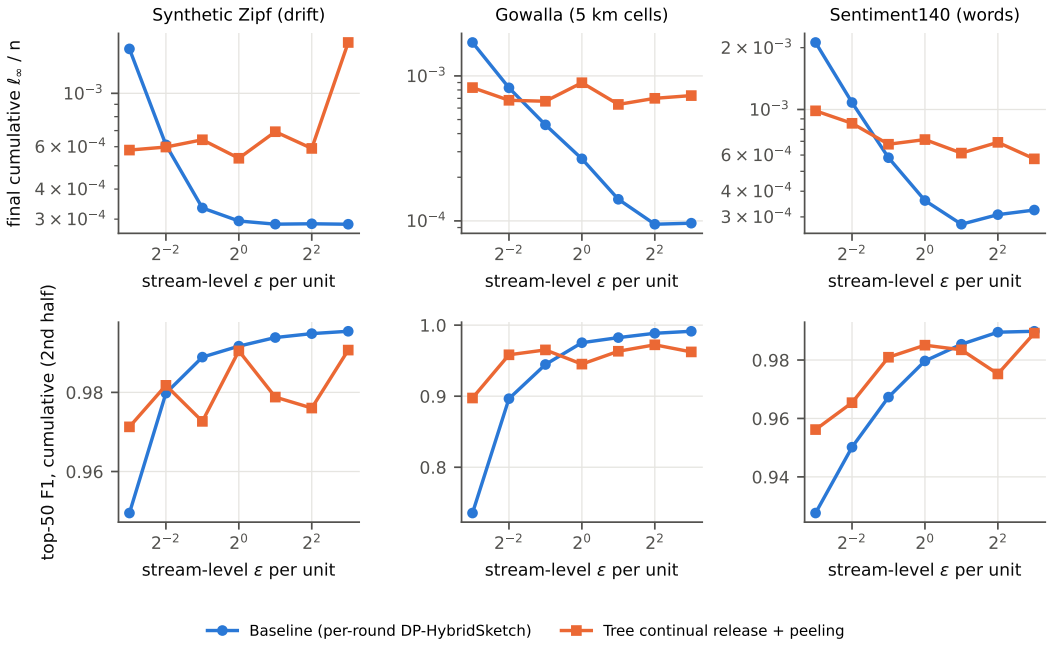


Fig. 5. Final cumulative ℓ_∞ (normalised, top) and second-half cumulative top-50 F1 (bottom) vs. stream-level ε.

TABLE VI

FINAL CUMULATIVE ℓ_∞ ERROR (COUNTS) VS. STREAM-LEVEL ε: PER-ROUND RELEASE (B) VS. TREE + PEELING (T). BOLD: LOWER.

	Synthetic		Gowalla		Sentiment140	
ε	B	T	B	T	B	T
0.125	6119	**2324**	4394	**2146**	2913	**1354**
0.25	2443	**2392**	2139	**1755**	1486	**1177**
0.5	**1335**	2565	**1188**	1728	**801**	931
1	**1179**	2149	**693**	2319	**495**	981
2	**1144**	2770	**364**	1643	**380**	843
4	**1148**	2361	**245**	1810	**423**	951
8	**1143**	6507	**249**	1890	**445**	790

VIII. CONCLUSION AND FUTURE WORK

We built a federated, privacy-preserving streaming heavy-hitter pipeline on Kafka, Flower, SecAgg and Grafana, reproduced three base papers in it and extended them for streams. The drift-aware variant recovers 3.7× faster and saves up to 61% upload; tree release with peeling halves cumulative error at small budgets. Future work: dropout-robust SecAgg, user-level accounting and DP candidate discovery without a dictionary.

REFERENCES

[1] J. Wu, W. Zhu, P. Kairouz, and V. Braverman, "Private federated frequency estimation: Adapting to the hardness of the instance," in *Proc. NeurIPS*, 2023.

[2] W. Zhu, P. Kairouz, B. McMahan, H. Sun, and W. Li, "Federated heavy hitters discovery with differential privacy," in *Proc. AISTATS*, PMLR 108, 2020.

[3] A. Gascón, P. Kairouz, Z. Sun, and A. T. Suresh, "Federated heavy hitter recovery under linear sketching," in *ICML Workshop on Federated Learning*, 2023.

[4] K. Bonawitz *et al.*, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM CCS*, 2017.

[5] C. Dwork, M. Naor, T. Pitassi, and G. N. Rothblum, "Differential privacy under continual observation," in *Proc. STOC*, 2010.

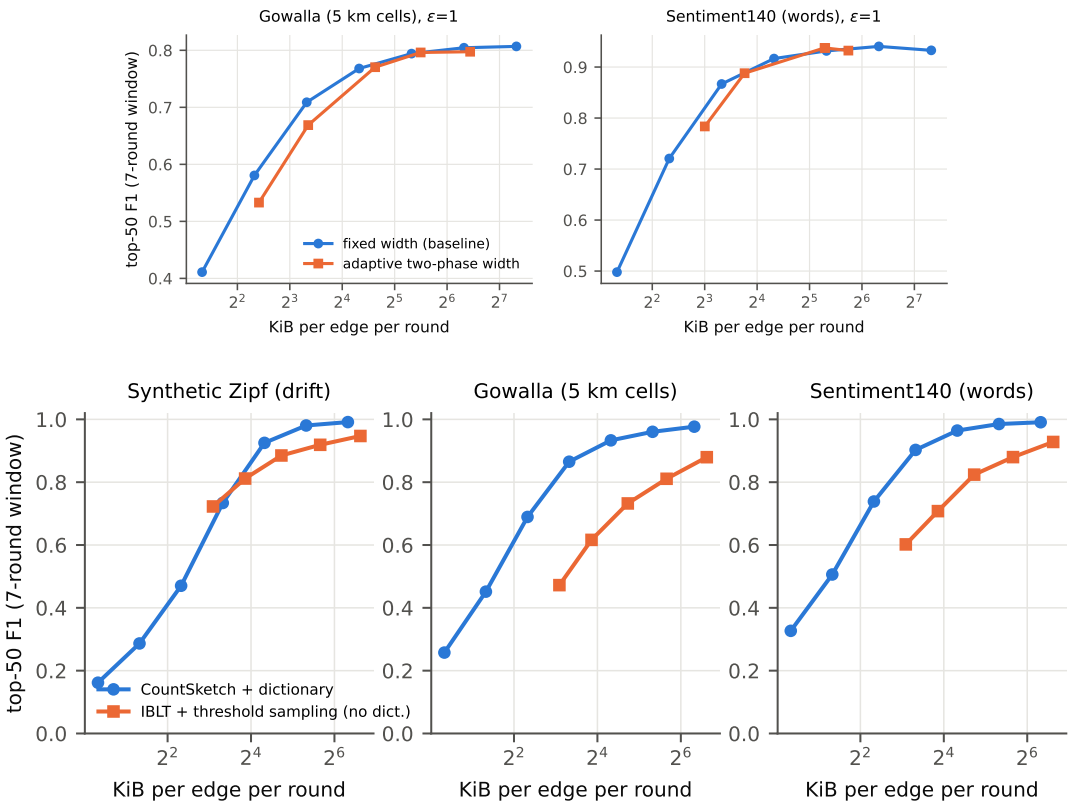


Fig. 6. Top: window F1 vs. upload, fixed vs. adaptive width ($\varepsilon = 1$). Bottom: no-DP heavy hitters, CountSketch+dictionary vs. IBLT.

[6] T.-H. H. Chan, E. Shi, and D. Song, "Private and continual release of statistics," *ACM Trans. Inf. Syst. Secur.*, vol. 14, no. 3, 2011.

[7] M. T. Goodrich and M. Mitzenmacher, "Invertible Bloom lookup tables," in *Proc. Allerton*, 2011.

[8] M. Charikar, K. Chen, and M. Farach-Colton, "Finding frequent items in data streams," in *Proc. ICALP*, 2002.

[9] D. J. Beutel *et al.*, "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.

[10] I. Mironov, "Rényi differential privacy," in *Proc. IEEE CSF*, 2017.